



Generación y Optimización de Código (80 horas)



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN



➤ Mejores prácticas de colaboración humano-IA en programación

- I. El modelo de colaboración efectiva*
- II. Los 10 principios de colaboración humano-IA*
- III. Workflow recomendado: Ciclo de desarrollo con IA*
- IV. Cuándo usar IA vs. programación manual*
- V. Errores comunes en la colaboración humano-IA*
- VI. Buenas prácticas avanzadas*



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

El modelo de colaboración efectiva



Si hablamos de tener una colaboración humana-IA no consiste en que la IA "haga el trabajo", sino en integrar sus capacidades dentro del proceso de desarrollo profesional.

El objetivo es que cualquier programador mantenga el control, el criterio y la responsabilidad, mientras la IA acelera tareas repetitivas, sugiere mejoras y ayuda a explorar alternativas.



4.1 El modelo de colaboración efectiva

La IA aporta:

- ☐ Velocidad
- ☐ Generación de boilerplate(código repetitivo)
- ☐ Conocimiento transversal de lenguajes
- ☐ Sugerencias de patrones comunes
- ☐ Documentación automática

■ 4.1 El modelo de colaboración efectiva

Que aportan los programadores:

- ☐ Comprensión del negocio
- ☐ Pensamiento crítico
- ☐ Arquitectura Ética y responsabilidad
- ☐ Detección de errores sutiles

4.1 El modelo de colaboración efectiva

Idea clave

La IA es un copiloto, no un piloto.

Dicho de otra manera, la IA propone y los programadores son los que evalúan el código generado, pero no solo en el pensamiento crítico, sino en la arquitectura, ética, responsabilidad, como al final los responsables finales del código de su calidad, de su seguridad y de su impacto.

El modelo de colaboración efectiva

4.1 El modelo de colaboración efectiva

Modelo mental incorrecto vs correcto

✗ Modelo incorrecto:

"La IA programa por mí"

✓ Modelo correcto:

"La IA acelera partes del proceso que yo diseño"

"LA IA PROGRAMA POR MÍ"

[Yo]



[IA] → [Código mágico]

Resultado:

- No entiendo el código
- No sé arreglarlo si falla
- Dependencia total
- Estancamiento profesional

"LA IA ACELERA PARTES DEL PROCESO QUE YO DISEÑO"

HUMANO (TÚ)

- Diseña la arquitectura
- Define los componentes
- Toma decisiones



IA (HERRAMIENTA)

- Genera código rápido
- Sugiere patrones
- Acelera tareas repetitivas



HUMANO (TÚ)

- Valida el código
- Prueba la funcionalidad
- Ajusta lo que no sirve



RESPONSABILIDAD (TUYA)

- El producto final es tuyo
- Los errores son tuyos
- El mérito también

4.1 El modelo de colaboración efectiva

Modelo mental incorrecto vs correcto

✗ Modelo incorrecto:

"La IA programa por mí"

✓ Modelo correcto:

"La IA acelera partes del proceso que yo diseño"

✗ Modelo Incorrecto

```
prompt = "Hazme una calculadora científica completa"
```

La IA te da 200 líneas que no entiendes

✓ Modelo Correcto

```
prompt = ""
```

Tú: "Voy a hacer una calculadora. Primero, funciones básicas:

- Suma
- Resta
- Multiplicación
- División

IA: Dame una función para suma con type hints"

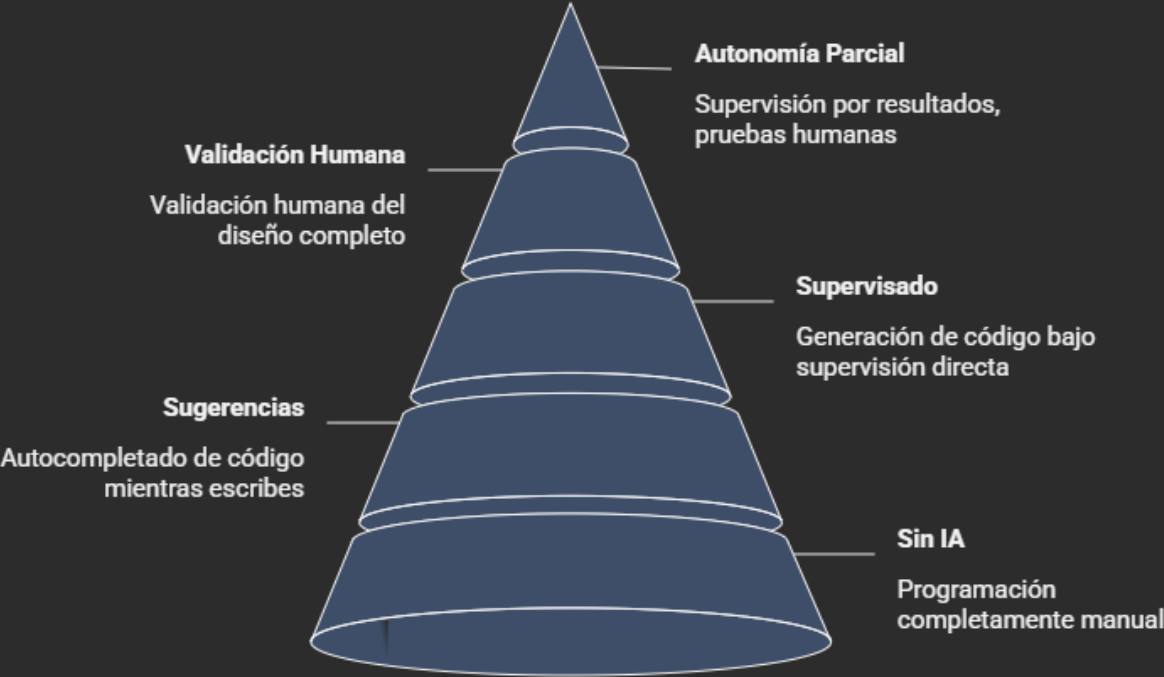
```
""
```

Tú entiendes, validas y decides qué sigue

4.1 El modelo de colaboración efectiva

Niveles de autonomía de la IA

Jerarquía de Autonomía de la IA en Programación



NIVELES DE AUTONOMÍA DE LA IA EN PROGRAMACIÓN		
Nivel	Descripción	Ejemplo
Nivel 0 Sin ayuda	Sin IA Código 100% propio	Todo manual "Lo escribo yo"
Nivel 1 Copilot	Sugerencias Autocompletado mientras escribes	GitHub Copilot "Sugiere mientras escribo"
Nivel 2 Supervisado	Generación bajo supervisión directa Revisas línea por línea	"Crea una función que valide email" y revisas cada línea
Nivel 3 Validación humana	Generación estructural Validación humana del diseño completo	"Dame la estructura de una clase Cache, yo reviso el diseño"
Nivel 4 Supervisada	Autonomía parcial Supervisión por resultados	"Genera el módulo completo, yo pruebo que funcione"

4.1 El modelo de colaboración efectiva

Jerarquía de Desarrollo de Habilidades



Made with Napkin

Nivel 4	▲	▲ PELIGRO (sin experiencia) • Solo si sabes lo que haces • Para tareas muy conocidas
Nivel 3		● IDEAL para aprender • Validas el diseño • Entiendes la estructura
Nivel 2		● PERFECTO para practicar • Revisas cada línea • Aprendes patrones
Nivel 1		● BUENO para productividad • Acelera escritura • No reemplaza conocimiento
Nivel 0		● APRENDIZAJE PURO • Para entender fundamentos

4.1 El modelo de colaboración efectiva

Mini caso práctico

Caso:

Startup que delega completamente el backend a IA sin revisión.

Resultado:

- Fallos de seguridad
- Código duplicado
- Arquitectura inconsistente
- Deuda técnica en 3 meses

Sirve para abrir debate.

EMPRESA: TechStart S.L.

FUNDADOR: Carlos, 22 años, "no necesita programadores"

MENTALIDAD: "La IA programa por mí"

PLAZO: Quiere el MVP en 2 semanas

4.1 El modelo de colaboración efectiva

Deuda Técnica por IA No Supervisada

Backend Delegado a IA

Backend completamente automatizado

Fallos de Seguridad

Vulnerabilidades de seguridad no detectadas

Código Duplicado

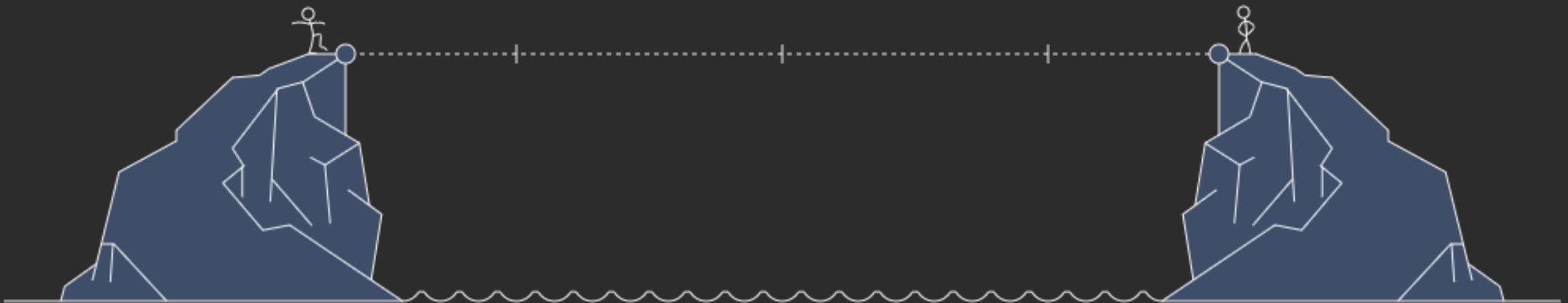
Código repetitivo y redundante

Arquitectura Inconsistente

Falta de estructura y coherencia

Deuda Técnica Acumulada

Backend inmanejable y costoso





4.1 El modelo de colaboración efectiva

Mini caso práctico: Semana 1



```
# Carlos pide a IA: "Sistema de autenticación completo"
# La IA genera:

def login(user, pass):
    # IA generó esto sin pensar en seguridad
    query = f"SELECT * FROM users WHERE user='{user}' AND pass='{pass}'"
    # 🚩 SQL INJECTION gratuita
    return ejecutar_query(query)

# Carlos: "¡Funciona! Qué bien, en 5 minutos lo tengo"
```



4.1 El modelo de colaboración efectiva

Mini caso práctico Semana 4: Las primeras señales

El código ya es un desastre:

archivo1.py

```
def validar_email(email):
```

```
    return "@" in email # ✅ Superficial
```

archivo2.py

```
def check_email(email):
```

```
    return "@" in email and "." in email # ❌ Duplicado
```

archivo3.py

```
def es_email_valido(email): # ❌ Tercera versión
```

```
    return re.match(r"...", email) # Diferente lógica
```

Misma funcionalidad, 3 implementaciones diferentes

4.1 El modelo de colaboración efectiva

Mini caso práctico Mes 3: El desastre

DIAGNÓSTICO TÉCNICO

FALLOS DE SEGURIDAD:

- 3 vulnerabilidades SQL injection
- Contraseñas en texto plano
- Tokens hardcodeados

CÓDIGO DUPLICADO:

- 5 funciones "validar_email" diferentes
- 3 clases "Usuario"
- 2 sistemas de login

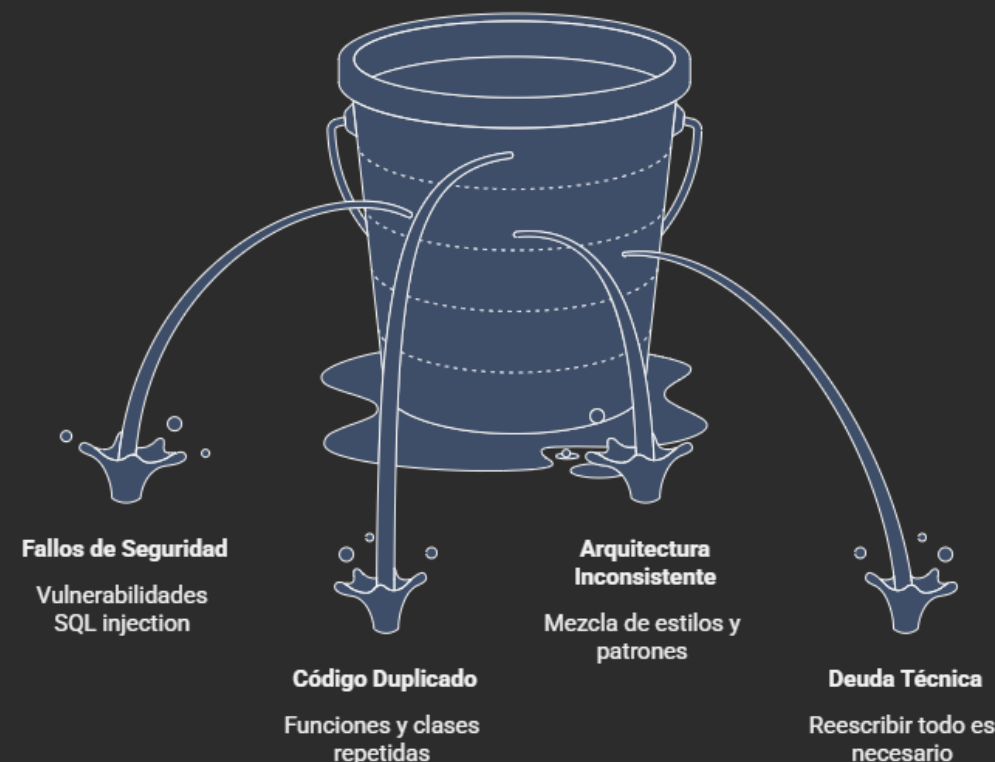
ARQUITECTURA INCONSISTENTE:

- Mezcla de estilos
- Sin patrones claros
- Dependencias circulares

DEUDA TÉCNICA:

- Refactorizar = reescribir todo
- 3 meses de trabajo perdido
- Contratar devs sale CARO

Diagnóstico Técnico: Un Desastre en Construcción



4.1 El modelo de colaboración efectiva

Mini caso práctico, La conversación final

- ✓ Carlos: "Necesito contratar dos developers senior"
- ✓ Inversor: "¿Pero no decías que la IA lo hacía todo?"
- ✓ Carlos: "Pensé que sí..."
- ✓ Developer senior: "Carlos, esto no es código, es un Frankenstein"
- ✓ Developer senior: "Vale 15.000€ y 2 meses arreglarlo"
- ✓ Carlos: "🤖 Mi startup..."

■ 4.1 El modelo de colaboración efectiva

Mini caso práctico,

ERROR 1: NIVEL DE AUTONOMÍA INADECUADO

Carlos usó Nivel 4 sin tener Nivel 0,1,2,3
Fue como darle un F1 a alguien sin carnet

ERROR 2: SIN VALIDACIÓN HUMANA

Nadie revisó el código
Nadie preguntó "¿esto está bien?"

ERROR 3: SIN DISEÑO PREVIO

No hubo arquitectura
Cada pieza se generó sin pensar en las demás

ERROR 4: RESPONSABILIDAD DELEGADA

"Si la IA lo hizo, el error es de la IA"
No, el error es de quien lo usa sin criterio

APRENDIZAJE:

- NIVEL 0-1: Aprende fundamentos
 - Escribe código manualmente
 - Entiende cada línea
- NIVEL 2: Practica con supervisión
 - Usa IA para tareas concretas
 - Revisa TODO
- NIVEL 3: Diseña tú, ejecuta IA
 - Tú haces la arquitectura
 - IA acelera implementación
- NIVEL 4: Solo cuando sepas
 - Tienes experiencia
 - Sabes lo que estás haciendo
 - Puedes detectar errores de IA

■ 4.2 Los 10 principios de colaboración humano-IA

Estos principios deben enseñarse como parte del workflow profesional.

- ◆ 1. Tú eres el arquitecto
- ◆ 2. Especifica el contexto
- ◆ 3. Itera y refina
- ◆ 4. Valida siempre
- ◆ 5. Aprende del código generado

■ 4.2 Los 10 principios de colaboración humano-IA

Estos principios deben enseñarse como parte del workflow profesional.

- ◆ 6. Combina herramientas
- ◆ 7. Documenta decisiones
- ◆ 8. Mantén tu criterio
- ◆ 9. Feedback continuo
- ◆ 10. Sigue aprendiendo

■ 4.2 Los 10 principios de colaboración humano-IA

◆ 1. Tú eres el arquitecto

La IA no decide la arquitectura ni las decisiones críticas.

La IA puede sugerir código, estructuras e incluso patrones, pero no entiende el contexto completo del sistema:

- Requisitos del cliente,
- Restricciones de seguridad,
- Escalabilidad,
- Mantenimiento a largo plazo,

Por eso, el rol del desarrollador no es "aceptar o rechazar sugerencias", sino **diseñar el sistema** y usar la IA como herramienta dentro de ese diseño.

4.2 Los 10 principios de colaboración humano-IA

◆ 1. Tú eres el arquitecto

✗ Mal enfoque

```
# Dejas que la IA decida la estructura
prompt = "Hazme una app de tareas"
# La IA genera algo genérico que no se adapta a tu caso
```

✓ Buen enfoque

```
# Tú diseñas, IA ejecuta
prompt = """
Voy a crear un gestor de tareas con esta estructura:
```

```
1. models/
   - Tarea.py (con título, descripción, fecha)
   - Usuario.py
```

```
2. controllers/
   - TareaController.py (CRUD básico)
```

```
3. views/
   - consola.py (menú interactivo)
```

```
Empieza por crear la clase Tarea con sus atributos
"""
```

```
# Tú decides la arquitectura
# La IA solo implementa piezas específicas
```

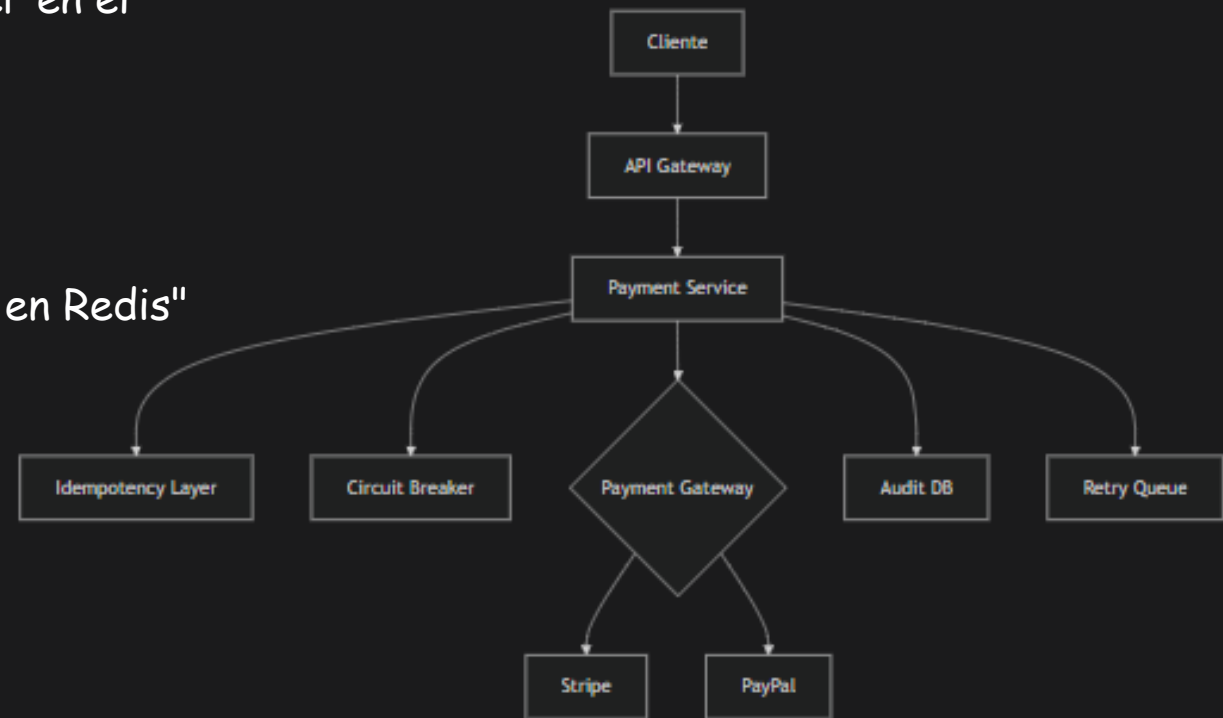
TÚ: "Yo sé a dónde voy"

IA: "Yo te ayudo a llegar más rápido"

Prompt para IA:

"Necesito ayuda implementando el patrón Circuit Breaker en el contexto de:

- Servicio de pagos que llama a múltiples gateways
- Timeout: 5 segundos por intento
- Fallback: Si Stripe falla, intentar PayPal
- Umbral: 5 fallos consecutivos abren el circuito
- Implementación en TypeScript con estado persistente en Redis"



// Prompt ingenuo: "Crea un sistema de pagos para e-commerce"

// IA podría sugerir:

```
app.post('/payment', async (req, res) => {  
  const payment = await stripe.charges.create({  
    amount: req.body.amount,  
    currency: 'usd',  
  });  
  res.json(payment);  
});
```

// ❌ Problemas: No maneja errores, sin logs, sin idempotencia, sin validación



Lo que NO hacer (delegar a la IA):



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL
DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL
SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

■ 4.2 Los 10 principios de colaboración humano-IA

◆ 2. Especifica el contexto

Cuanto más contexto, mejores sugerencias.

La IA no "adivina" lo que quieres: razona a partir de la información que le das.

Cuando el programador ofrece poco contexto, la IA rellena huecos con suposiciones.

Cuando el programador ofrece mucho contexto, la IA trabaja con precisión quirúrgica.

Este principio es clave para enseñar a programadores a comunicarse con la IA como profesionales.

❌ POCO Contexto (Típico de principiantes)

```
// Prompt: "Haz una función para usuarios"
// Respuesta IA (genérica y probablemente inútil):
function processUser(user) {
  return user;
}
// ❌ No sabe qué hacer, así que da lo más básico
```

✅ MUCHO Contexto (Profesional)

```
// Prompt: "Crea una función para validar usuarios en registro:
// - Usamos TypeScript con Node.js 18
// - El objeto User tiene: email, password, age, country
// - Validaciones: email formato válido, password > 8 chars con mayúscula y número
// - Edad mínima: 13, máxima: 120
// - Países permitidos: ['US', 'MX', 'ES', 'CO']
// - Retornar: { success: boolean, errors: string[] }
// - Usar expresiones regulares para email
// - Exportar como validateUserRegistration"
```

❌ POCO Contexto (Típico de principiantes)

```
prompt = "Valida un email" # Demasiado genérico
```

✅ MUCHO Contexto (Profesional)

```
prompt = """
```

Contexto: App de registro para una universidad española

Requisitos:

- Emails deben ser institucionales (@universidad.es)
- Máximo 50 caracteres
- No permitir emails temporales (hotmail, gmail)
- Debe validar también emails de profesores (@prof.universidad.es)
- Integraremos con base de datos existente

Implementa la función de validación considerando:

- Manejo de None y strings vacíos
 - Quitar espacios
 - Case insensitive
 - Tests para casos normales y edge cases
- ```
"""
```

## ■ 4.2 Los 10 principios de colaboración humano-IA

### ◆ 3. Itera y refina

La IA funciona mejor por pasos, no con prompts gigantes.

En lugar de: "Hazme una API completa con autenticación, base de datos, tests, documentación y Docker."

Mejor:

Primero: "Diseña las rutas principales de una API REST para gestión de usuarios (solo estructura, sin implementación)."

Luego: "Ahora genera los controladores en TypeScript para estas rutas, usando buenas prácticas."

Después: "Crea tests unitarios para estos controladores con Jest."

Finalmente:

"Genera un README que explique cómo usar esta API."

Cada paso es más controlable, más revisable y más fácil de corregir.

## Proceso de refinamiento

Iteración 1: 5 líneas → Función básica

Iteración 2: 20 líneas → Lógica de negocio

Iteración 3: 50 líneas + tests → Sistema robusto

# Iteración 1 - Básico

```
prompt = "Función para calcular precio con IVA"
```

```
IA devuelve: precio * 1.21
```

# Iteración 2 - Añadir contexto

```
prompt = """
```

Ahora mejórala:

- El IVA varía por producto (4%, 10%, 21%)
- Productos infantiles tienen 4%
- Libros tienen 4% también
- Electrónica tiene 21%
- Añade tests

```
"""
```

# Iteración 3 - Refinar más

```
prompt = """
```

Perfecto, ahora:

- Si el producto es de segunda mano, 50% descuento
- Clientes VIP tienen 10% adicional
- Redondea a 2 decimales
- Maneja errores de tipo

```
"""
```

## ■ 4.2 Los 10 principios de colaboración humano-IA

### ◆ 4. Valida siempre

Nunca aceptar código sin entenderlo.

Este principio es uno de los más importantes en la colaboración humano-IA.

La IA puede generar código muy convincente, bien formateado y aparentemente correcto... pero eso no significa que sea seguro, eficiente o adecuado para tu proyecto.

Aceptar código sin entenderlo convierte al programador en un "pegador de snippets", no en un profesional.

```
def dividir(a, b):
 return a / b
```

## Lo que NO debes hacer

```
ERROR: Copiar y pegar sin pensar
codigo_ia = """
def procesar_pago(tarjeta):
 # Código que parece funcionar
 return ejecutar_pago(tarjeta)
"""

No viste que falta validación de seguridad
```

## Lo que SÍ debes hacer

```
def procesar_pago(tarjeta):
 # Revisar: ¿Valida formato?
 # Revisar: ¿Maneja errores?
 # Revisar: ¿Logs?
 # Revisar: ¿Inyección SQL?
 return ejecutar_pago(tarjeta)

PASO 2: Tests manuales
print(procesar_pago("1234-5678-9012-3456")) # ¿Funciona?
print(procesar_pago("invalido")) # ¿Falla bien?
print(procesar_pago(None)) # ¿Explota?

PASO 3: Tests automatizados
def test_procesar_pago():
 assert procesar_pago("1234-5678-9012-3456") == True
 assert procesar_pago("invalido") == False
 assert procesar_pago(None) == False
 print("✅ Tests pasan")

PASO 4: Validar con requisitos
✅ Cumple formato
✅ Cumple seguridad
✅ Cumple manejo errores
```

## ■ 4.2 Los 10 principios de colaboración humano-IA

### ◆ 5. Aprende del código generado

La IA puede enseñar patrones, estilos y técnicas.

Uno de los mayores beneficios de trabajar con IA no es solo que genere código rápidamente, sino que expone al programador a nuevas formas de resolver problemas.

Cada fragmento de código generado es una oportunidad para aprender:

- cómo estructurar funciones
- cómo aplicar patrones de diseño
- cómo escribir código más limpio
- cómo documentar correctamente
- cómo mejorar la legibilidad y mantenibilidad

La IA se convierte así en un mentor técnico, disponible 24/7, que muestra alternativas, estilos y enfoques que quizá el programador no conocía.

# Transformar código en conocimiento



Código IA → Preguntas → Investigación  
→ Mejora → Conocimiento propio

# 1. Código generado por IA

```
prompt = "Ordenar lista de diccionarios por clave"
codigo = """
def ordenar_por_clave(lista, clave):
 return sorted(lista, key=lambda x: x[clave])
"""
```

# 2. Preguntas que te haces:

```
"""
? ¿Qué es lambda?
? ¿Cómo funciona key?
? ¿Por qué sorted y no sort?
? ¿Qué pasa si la clave no existe?
"""
```

# 3. Profundizar

```
Buscas: "Python lambda explicación"
Buscas: "Python sorted key function"
Buscas: "Python manejo errores key dict"
```

# 4. Mejorar con lo aprendido

```
def ordenar_por_clave_seguro(lista, clave):
 """
 Versión mejorada: aprendí que puede fallar
 """
 try:
 return sorted(lista, key=lambda x: x.get(clave, ''))
 except (TypeError, AttributeError):
 return lista # o lanzas error controlado
```

## ■ 4.2 Los 10 principios de colaboración humano-IA

### ◆ 6. Combina herramientas

- Copilot para código, (Yo prefiero Deep o Claude)
- ChatGPT para explicaciones, (Yo prefiero Napkin)
- Perplexity (para investigar)

En el trabajo profesional, ningún desarrollador usa una sola herramienta de IA.

Cada asistente tiene puntos fuertes distintos, y aprender a combinarlos es una habilidad clave para trabajar de forma eficiente, rápida y con calidad.

**La idea es simple:** usa cada IA para lo que mejor sabe hacer.



## PRINCIPIO 6: COMBINA HERRAMIENTAS No solo ChatGPT

# Situación: Necesitas un sistema completo

#  GitHub Copilot (para escribir)

# Escribes: "def calcular\_impuestos"

# Copilot sugiere automáticamente

#  ChatGPT (para diseño)

prompt = "Dame 3 formas de estructurar este sistema"

# IA te da opciones, tú eliges

#  Claude (para refactorizar)

prompt = """

Este código funciona pero es feo. Dame 5 formas de mejorarlo:

- Más legible
- Más eficiente
- Más pythonico

"""

#  Perplexity (para investigar)

"mejores prácticas python para manejo de errores 2024"

"patrones de diseño para sistemas de caché"

#  Tu IDE (para ejecutar, depurar, testear)

# Nada reemplaza ejecutar el código y ver qué pasa

## ■ 4.2 Los 10 principios de colaboración humano-IA

### ◆ 7. Documenta decisiones

Indica qué partes fueron generadas por IA y por qué.

En un flujo de trabajo profesional, no basta con usar IA:

hay que dejar rastro de cómo, cuándo y por qué se ha utilizado.

Documentar las decisiones tomadas con ayuda de IA es una práctica esencial para:

- mantener transparencia
- facilitar auditorías
- mejorar la trazabilidad del código
- evitar problemas legales o éticos
- permitir que otros desarrolladores entiendan el proceso

La IA no sustituye la responsabilidad del programador, y documentar su uso es parte de esa responsabilidad.

```
❌ MAL: Código sin contexto
```

```
def calcular(x, y):
 return x * 0.21 + y * 0.04
```

```
✅ BIEN: Decisiones documentadas
```

```
def calcular_impuestos(base, tipo_producto):
 """
```

```
 Calcula impuestos según producto y región
```

```
 DECISIONES DE NEGOCIO:
```

- IVA 21% para electrónica (decisión 2024-01)
- IVA 4% para libros (ley cultura 2023)
- Descuento 50% segunda mano (política empresa)

```
 RESTRICCIONES TÉCNICAS:
```

- Base de datos legacy usa céntimos (entero)
- API externa espera euros (float)

```
 CAMBIOS FUTUROS:
```

- TODO: parametrizar por región (UE)
- TODO: añadir impuesto digital 2025

```
 Por qué no usamos Decimal:
```

- Compatibilidad con sistema antiguo
- Velocidad sobre precisión (no financiero)

```
 """
```

```
Código aquí
```

# PRINCIPIO 7: DOCUMENTA DECISIONES

## Por qué documentar

# PRINCIPIO 7: DOCUMENTA DECISIONES

Plantilla para  
documentar

"""

FUNCIÓN: [nombre]

CREADA: [fecha] con IA (ChatGPT)

MOTIVO: [problema que resuelve]

DECISIONES:

- [decisión 1] porque [razón]
- [decisión 2] porque [razón]

LIMITACIONES:

- [limitación conocida]
- [caso no cubierto]

CAMBIOS:

- v2: añadido manejo errores (por fallo en prod)
- v1: versión inicial

"""

## ■ 4.2 Los 10 principios de colaboración humano-IA

### ◆ 8. Mantén tu criterio

La IA no sustituye el juicio profesional.

La IA puede generar código, sugerencias, explicaciones y alternativas... pero no entiende el proyecto como tú.

No conoce al cliente, no conoce las restricciones reales, no sabe qué decisiones ya tomó el equipo ni qué compromisos existen entre rendimiento, seguridad y tiempo.

Por eso, el criterio profesional del desarrollador sigue siendo imprescindible.

# EJEMPLO 1: IA propone solución insegura

```
prompt = "Validar contraseña"
```

```
ia_responde = ""
```

```
def validar_password(pass):
```

```
 # MAL: Guardar en texto plano
```

```
 return pass == "admin123"
```

```
"""
```

```
TÚ sabes que eso es inseguro
```

```
TÚ propones hashing
```

# EJEMPLO 2: IA propone código ineficiente

```
ia_responde = ""
```

```
def buscar_producto(lista, nombre):
```

```
 for i in range(len(lista)): # MAL: Ineficiente
```

```
 if lista[i]["nombre"] == nombre:
```

```
 return lista[i]
```

```
"""
```

```
TÚ sabes que un diccionario es mejor
```

# EJEMPLO 3: IA inventa bibliotecas

```
ia_responde = ""
```

```
import superlib # Esta librería NO EXISTE
```

```
"""
```

```
TÚ sabes verificar documentación real
```

# PRINCIPIO 8: MANTÉN TU dicampus CRITERIO

## La IA se equivoca (y mucho)

IA: "Usa esta librería mágica"

TÚ: "¿Existe? ¿Está mantenida?  
¿Es segura?"

IA: "Este código es óptimo"

TÚ: "¿Para 10 o 10 millones de datos?"

IA: "Hazlo así"

TÚ: "¿Cumple nuestros requisitos?"



## ■ 4.2 Los 10 principios de colaboración humano-IA

### ◆ 8. Mantén tu criterio

#### 🧠 ¿Qué significa “mantener tu criterio”?

- No aceptar sugerencias solo porque “suenan bien”.
- Evaluar si la solución encaja con la arquitectura del proyecto.
- Revisar si cumple requisitos funcionales y no funcionales.
- Detectar riesgos que la IA no ve (seguridad, rendimiento, deuda técnica).
- Elegir entre varias alternativas según experiencia y contexto.

## ■ 4.2 Los 10 principios de colaboración humano-IA

### ◆ 9. Feedback continuo

Indica a la IA qué sugerencias son útiles y cuáles no.

La IA aprende dentro de la conversación según el feedback que le das.

No aprende a largo plazo, pero sí ajusta su comportamiento en tiempo real.

Si no le dices qué te sirve y qué no, seguirá generando respuestas genéricas.

Si le das feedback claro, se vuelve mucho más precisa.

## 4.2 Los 10 principios de colaboración humano-IA

### 9. Feedback continuo

#### ¿Qué tipo de feedback funciona?

- "Esta solución es demasiado compleja, hazla más simple."
- "No uses clases, prefiero funciones puras."
- "Evita dependencias externas."
- "Esto no sigue el patrón que estamos usando."
- "Bien, pero añade validaciones."
- "Esto no es lo que necesito, te explico mejor..."

# PRINCIPIO 9: FEEDBACK CONTINUO

## Enseña a la IA a mejorar

```
feedbacks = [
 "✅ Bueno: [lo que está bien]",
 "❌ Malo: [lo que falla]",
 "🟡 Mejorable: [lo que puede mejorar]",
 "📊 Contexto: [datos reales de uso]",
 "🎯 Objetivo: [qué queremos lograr]"
]
```

```
Iteración con feedback explícito

Turno 1
tú: "Crea función que lea CSV"
ia: "aquí tienes read_csv() básico"

Turno 2 - Feedback
tú: ""
La función funciona pero:
1. No maneja archivos grandes (memory error)
2. Falla si hay caracteres especiales
3. No tiene logging

Mejórala con estas consideraciones
""

Turno 3 - Más feedback
tú: ""
Ahora mejor, pero:
- Los errores no son descriptivos
- Falta typing
- Añade docstring
""

Turno 4 - Refinado
tú: ""
Perfecto, ahora:
- Hazla asíncrona
- Añade timeout
- Integra con nuestro sistema de logging
""
```

## 4.2 Los 10 principios de colaboración humano-IA

### ◆ 10: SIGUE APRENDIENDO

La IA evoluciona, tú también

# 2023: Usabas ChatGPT básico  
prompt = "código python"

# 2024: Aprendiste a dar contexto  
prompt = "python función asíncrona con typing"

# 2025: Sabes exactamente qué pedir  
prompt = ""

Arquitectura hexagonal con:

- Domain models inmutables
- CQRS para consultas
- Event sourcing para auditoría
- Tests de integración con docker

Documentación automática con mkdocs

# TÚ + IA: Equipo Perfecto para la Programación



## PRINCIPIO 10: SIGUE APRENDIENDO

### Plan de evolución personal

```
aprendizaje = {
 "fase_1": "Pedir código simple y entenderlo",
 "fase_2": "Refinar prompts con contexto",
 "fase_3": "Diseñar arquitectura propia",
 "fase_4": "Revisar y mejorar código IA",
 "fase_5": "Combinar múltiples IAs",
 "fase_6": "Automatizar con IA pero supervisar",
 "meta": "Ser mejor que la IA en diseño, IA mejor en ejecución"
```

### ■ 4.3 Workflow recomendado: Ciclo de desarrollo con IA

Este es el flujo profesional.

- ◆ 1. Diseño (Humano)
- ◆ 2. Scaffolding (IA)
- ◆ 3. Implementación (Humano + IA)
- ◆ 4. Refactorización (IA + Humano)
- ◆ 5. Testing (Humano + IA)
- ◆ 6. Revisión (Humano)
- ◆ 7. Documentación (IA + Humano)

## 4.3

### Workflow

recomendado:

## Ciclo de desarrollo con IA



## 4.3 Workflow recomendado: Ciclo de desarrollo con IA

| Fase            | Desarrollo Clásico | Desarrollo con IA     | Diferencia                         |
|-----------------|--------------------|-----------------------|------------------------------------|
| Diseño          | Humano (días)      | Humano (horas)        | IA acelera investigación previa    |
| Scaffolding     | Manual tedioso     | Automático (segundos) | IA genera estructura base          |
| Implementación  | Todo manual        | IA para boilerplate   | 70% menos código escrito           |
| Refactorización | Manual o tools     | IA sugiere mejoras    | Descubres patrones que no conocías |
| Testing         | Escribes tests     | IA genera casos base  | Cubres más casos rápido            |
| Revisión        | Humano (lento)     | IA pre-revisa         | Menos errores obvios               |
| Documentación   | Odiosa, no se hace | IA la genera          | iPor fin hay docs!                 |

## 4.3 Workflow

recomendado:

Ciclo de desarrollo  
con IA

```
DESARROLLO CLÁSICO
```

```
tareas = [
 "Escribir estructura básica", # 2 horas
 "Implementar validaciones", # 3 horas
 "Escribir tests", # 2 horas
 "Documentar", # 1 hora
 "Revisar errores tontos", # 1 hora
]
```

```
total = 9 horas
```

```
errores_humanos = 5 # Tipos, olvidos, etc.
```

```
DESARROLLO CON IA
```

```
tareas = [
 "Diseñar solución (TÚ)", # 1 hora
 "IA genera estructura", # 1 minuto
 "Revisar y ajustar (TÚ)", # 1 hora
 "IA genera tests base", # 1 minuto
 "Completar casos edge (TÚ)", # 1 hora
 "IA sugiere mejoras", # 1 minuto
 "Validar (TÚ)", # 1 hora
 "IA genera documentación", # 1 minuto
]
```

```
total = 4 horas + 5 minutos
```

```
errores_ia = 2 # (TÚ los detectas)
```

## 4.3 Workflow recomendado: Ciclo de desarrollo con IA

### ◆ 1. Diseño (Humano)

# TÚ: Diseñas el sistema

|||||

Sistema de tareas con:

- Usuarios pueden crear tareas
- Tareas tienen: título, descripción, fecha, estado
- Cada usuario ve SOLO sus tareas
- Guardar en SQLite
- Interfaz por consola

|||||

## 4.3 Workflow recomendado: Ciclo de desarrollo con IA

### ◆ 2: SCAFFOLDING (IA)

# TÚ: "Genera estructura base para sistema de tareas"

prompt = ""

Crea la estructura de carpetas y archivos base para un sistema de tareas en Python:

- models/tarea.py
- models/usuario.py
- controllers/tarea\_controller.py
- controllers/usuario\_controller.py
- views/consola.py
- database/db.py

Genera el esqueleto de cada archivo con imports y clases vacías

|||||

## 4.3 Workflow recomendado: Ciclo de desarrollo con IA

### 2: SCAFFOLDING (IA) Estructura de Proyecto de Software

models/tarea.py

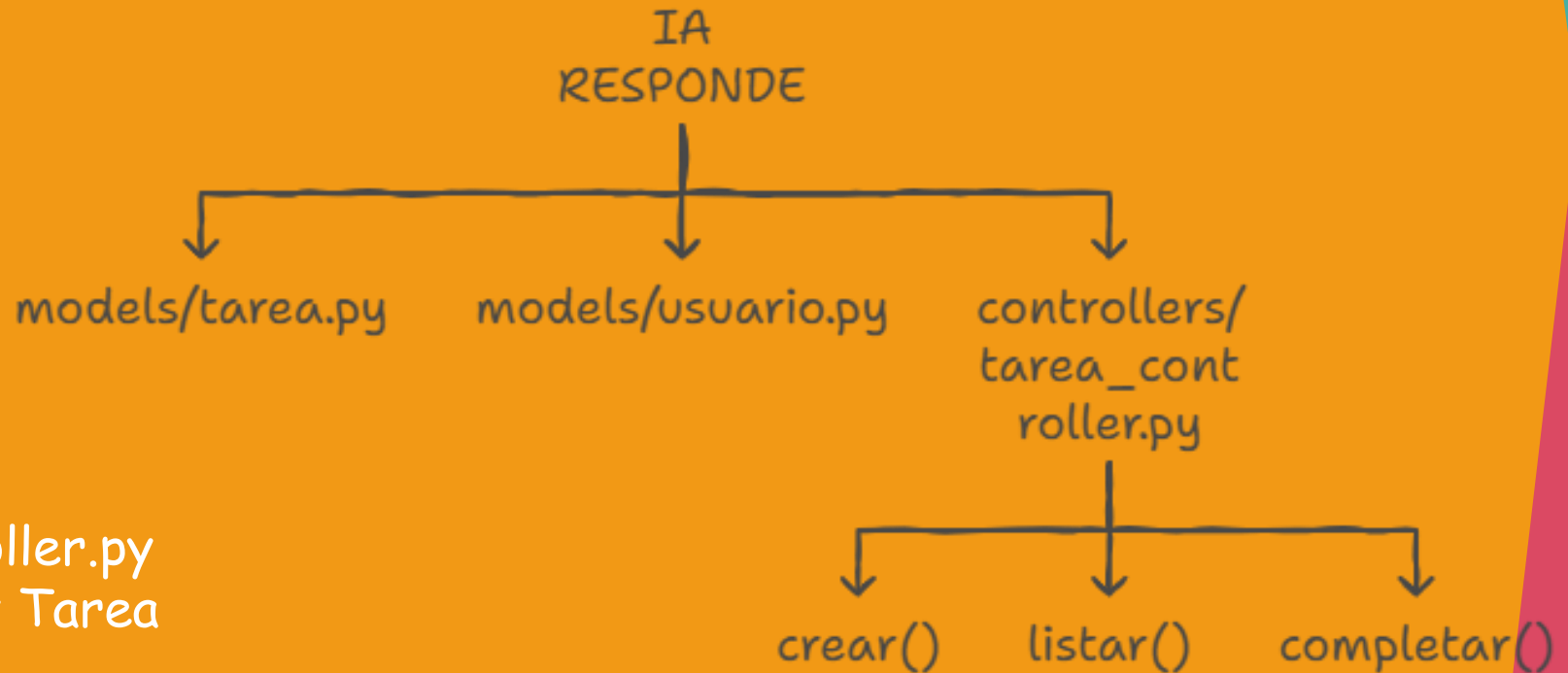
```
class Tarea:
 def __init__(self):
 pass
```

models/usuario.py

```
class Usuario:
 def __init__(self):
 pass
```

controllers/tarea\_controller.py

```
from models.tarea import Tarea
```



Made with Napkin

## 4.3 Workflow recomendado: Ciclo de desarrollo con IA

### ◆ 3: IMPLEMENTACIÓN (Humano + IA)

# TÚ: "Implementa la **clase Tarea** con atributos"

prompt = """

Completa la clase Tarea con:

- título (str)
- descripción (str)
- fecha\_creacion (datetime)
- completada (bool)
- usuario\_id (int)

Añade \_\_repr\_\_ y validación básica

"""

# IA genera el código, TÚ lo revisas y ajustas

# TÚ: "Ahora el método **crear\_tarea** en el controlador"

prompt = """

En TareaController, implementa crear(titulo, desc, usuario\_id) que:

1. Valide título no vacío
2. Cree instancia de Tarea
3. Guarde en base de datos (usa db.py)
4. Retorne la tarea creada

"""

## 4.3 Workflow recomendado: Ciclo de desarrollo con IA

### 4: REFACTORIZACIÓN (IA + Humano)

# TÚ: "Este código funciona, ¿cómo mejorarlo?"

```
codigo_actual = """
def crear_tarea(titulo, desc, user_id):
 if titulo == "":
 return None
 tareas = Tarea(titulo, desc, user_id)
 db.guardar(tarea)
 return tarea
"""
```

prompt = """

Sugiere 3 mejoras para este código considerando:

- Manejo de errores más robusto
- Validaciones completas
- Tipado
- Logging

"""

# IA SUGIERE:

"""

1. Usar Optional y excepciones
2. Añadir validación de longitud
3. Logging de operaciones
4. Type hints

"""

# TÚ decides qué implementar

## 4.3 Workflow recomendado: Ciclo de desarrollo con IA

### 5: TESTING (Humano + IA)

# TÚ: "Genera tests para la función crear\_tarea"

prompt = """

Crea tests con pytest para crear\_tarea considerando:

- Caso normal
- Título vacío
- Título muy largo
- Usuario no existe
- Descripción None

"""

# IA genera tests base

```
def test_crear_tarea_normal():
 tarea = controlador.crear_tarea
 ("Comprar pan", "Pan integral", 1)
 assert tarea.titulo == "Comprar pan"
 assert not tarea.completada
```

```
def test_crear_tarea_titulo_vacio():
 with pytest.raises(ValueError):
 controlador.crear_tarea("", "desc", 1)
```

# TÚ ejecutas y ajustas

## ■ 4.3 Workflow recomendado: Ciclo de desarrollo con IA

### ◆ 6: REVISIÓN (Humano 100%)

# TÚ revisas MANUALMENTE:

checklist = [

- "✓ ¿Cumple requisitos?",
- "✓ ¿Maneja errores?",
- "✓ ¿Es seguro? (SQL injection)",
- "✓ ¿Tipado correcto?",
- "✓ ¿Tests pasan?",
- "✓ ¿Rendimiento aceptable?",
- "✓ ¿Mantiene estándares del equipo?",

]

# Solo TÚ apruebas el código

if all(checklist):

print("✓ Código aprobado")

else:

print("✗ Revisar: puntos pendientes")

## 4.3 Workflow recomendado: Ciclo de desarrollo con IA

### 7: DOCUMENTACIÓN (IA + Humano)

# TÚ: "Genera documentación para este módulo"

prompt = """

Crea docstring para el módulo

**tarea\_controller** explicando:

- Propósito del controlador
- Métodos principales
- Ejemplos de uso
- Excepciones que lanza

"""

# IA genera borrador

"""

Módulo: **tarea\_controller**

Propósito: Gestiona operaciones CRUD de tareas

Métodos:

**crear\_tarea**(titulo, descripcion, usuario\_id)  
Crea nueva tarea para un usuario

**listar\_tareas**(usuario\_id, solo\_pendientes=False)  
Lista tareas de un usuario

Ejemplos:

>>> tc = TareaController()

>>> tarea = tc.crear\_tarea("Estudiar", "Python IA", 1)

Excepciones:

- ValueError: datos inválidos
- UsuarioNoExisteError: usuario\_id no encontrado

"""

# TÚ revisas, completas y formateas

# Proceso de Desarrollo con IA y Humano



Puntos donde el control humano es

**OBLIGATORIO**



**MOMENTOS CRÍTICOS**

## El modelo de colaboración efectiva

### 4.3 Workflow recomendado: Ciclo de desarrollo con IA

```
def aprobar_para_produccion(codigo):
 """
```

```
 Checklist que SOLO el humano puede validar
 """
```

```
 preguntas = [
 # NEGOCIO
```

#### # NEGOCIO

```
 "¿Resuelve el problema real?",
 "¿Cumple con los requisitos?",
 "¿Añade valor al usuario?",
```

#### # SEGURIDAD

```
 "¿Datos sensibles protegidos?",
 "¿Autenticación robusta?",
 "¿Prevención de ataques comunes?",
```

#### # CALIDAD

```
 "¿Código mantenible?",
 "¿Tests suficientes?",
 "¿Documentación clara?",
```

#### # RESPONSABILIDAD

```
 "¿Entiendo cada línea?",
 "¿Puedo explicarlo a mi equipo?",
 "¿Acepto las consecuencias de bugs?"
```

```
]
```

## 4.3 Workflow recomendado: Ciclo de desarrollo con IA



"La IA sugiere, el humano decide"

"La IA acelera, el humano valida"

"La IA escribe, el humano responde"

"La IA propone, el humano aprueba"

"La IA no tiene responsabilidad, el humano sí"

### PUNTO CLAVE:

El código generado por IA sin revisión humana es como un coche sin frenos: puede funcionar... hasta que no.



## 4.4 Cuándo usar IA vs. programación manual

| Escenario         | IA                 | Manual                       |
|-------------------|--------------------|------------------------------|
| CRUD básico       | ✓ Ideal            | ✗ Ineficiente                |
| Algoritmo crítico | ⚠ Asistencia       | ✓ Validación exhaustiva      |
| Código legacy     | ✓ Modernización    | ⚠ Solo si conoces el dominio |
| Seguridad         | ✗ Riesgo alto      | ✓ Auditoría manual           |
| Prototipado       | ✓ Velocidad máxima | ✗ Muy lento                  |
| Aprendizaje       | ✓ Tutor            | ✓ Para dominar conceptos     |

## 4.4 Cuando NO usar IA (Ética y Legal)

### ! ESCENARIOS PROHIBIDOS O DE ALTO RIESGO



#### 🚫 DATOS SENSIBLES O PERSONALES

- NO pegues emails reales, DNI, contraseñas en prompts
- NO subas bases de datos completas a ChatGPT
- ¿Qué pasa? Tus datos pueden usarse para entrenar modelos
- SOLUCIÓN: Anonimizar, usar versiones locales (Llama, etc.)

#### 🚫 CÓDIGO CON LICENCIA RESTRICTIVA

- NO pidas a IA que reproduzca código con copyright
- NO uses IA para eludir licencias (GPL, etc.)
- ¿Qué pasa? Problemas legales, demandas
- SOLUCIÓN: Conoce las licencias, usa código original

#### 🚫 SISTEMAS CRÍTICOS DE SEGURIDAD

- NO confíes en IA para autenticación bancaria
- NO uses IA para cifrado sensible
- ¿Qué pasa? Vulnerabilidades que IA no detecta
- SOLUCIÓN: Auditoría experta obligatoria

#### 🚫 TOMA DE DECISIONES SIN SUPERVISIÓN

- NO dejes que IA decida arquitectura por ti
- NO aceptes código sin entenderlo
- ¿Qué pasa? Deuda técnica, sistemas inmantenibles

- SOLUCIÓN: Tú mandas, IA obedece

#### 🚫 CUANDO NO ENTIENDES EL CÓDIGO

- NO uses código que no puedes explicar
- NO pongas en producción lo que no entiendes
- ¿Qué pasa? Bugs imposibles de arreglar
- SOLUCIÓN: Aprende primero, usa IA después





## 4.4 Cuando NO usar IA (Ética y Legal)



### ESCENARIOS PROHIBIDOS O DE ALTO RIESGO



```
def puedo_usar_IA(escenario):
 """
 Checklist antes de usar IA en producción
 """

 preguntas = [
 "¿Hay datos personales reales?",
 "¿Código con copyright?",
 "¿Sistema crítico de seguridad?",
 "¿Entiendo lo que voy a pedir?",
 "¿Puedo revisar el código generado?",
 "¿Asumo responsabilidad legal?",
]

 for p in preguntas:
 if p == "¿Hay datos personales reales?" and respuesta == "sí":
 return "🚫 NO: Anonimiza primero"

 return "✅ Puedes continuar con precaución"
```

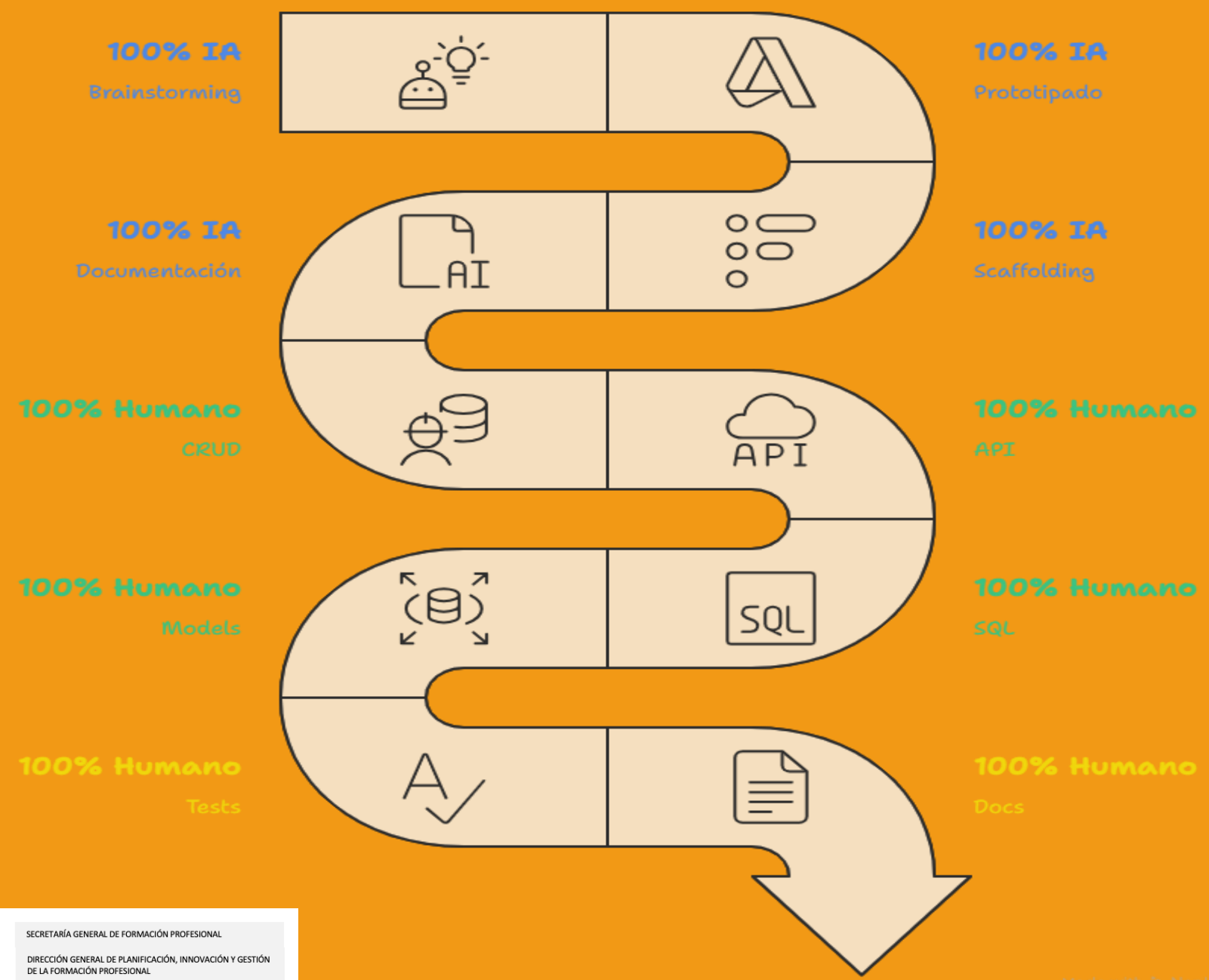


| Categoría    | Escenario                    | % IA | % Manual | Estrategia Recomendada                        |
|--------------|------------------------------|------|----------|-----------------------------------------------|
| 🟦 INICIO     |                              |      |          |                                               |
|              | Idea inicial / Brainstorming | 80%  | 20%      | IA para explorar opciones, humano filtra      |
|              | Definición de requisitos     | 10%  | 90%      | Humano con experiencia, IA solo documenta     |
|              | Diseño de arquitectura       | 30%  | 70%      | IA sugiere patrones, humano decide            |
|              | Elección de tecnología       | 20%  | 80%      | IA compara, humano según contexto             |
|              |                              |      |          |                                               |
| 🟡 DESARROLLO |                              |      |          |                                               |
|              | Modelos de datos             | 60%  | 40%      | IA genera base, humano ajusta reglas negocio  |
|              | Endpoints API REST           | 70%  | 30%      | IA boilerplate, humano valida                 |
|              | Validaciones complejas       | 40%  | 60%      | IA casos base, humano edge cases              |
|              | Lógica de negocio            | 20%  | 80%      | Humano diseña, IA implementa bajo supervisión |
|              | Consultas SQL                | 50%  | 50%      | IA escribe, humano optimiza                   |
|              | Conexión a BBDD              | 80%  | 20%      | IA patrón estándar, humano credenciales       |

| Categoría                                                                                       | Escenario                | % IA | % Manual | Estrategia Recomendada                      |
|-------------------------------------------------------------------------------------------------|--------------------------|------|----------|---------------------------------------------|
|  CALIDAD        |                          |      |          |                                             |
|                                                                                                 | Tests unitarios          | 60%  | 40%      | IA genera, humano completa edge cases       |
|                                                                                                 | Tests integración        | 30%  | 70%      | Humano diseña flujos, IA código repetitivo  |
|                                                                                                 | Debugging errores        | 40%  | 60%      | IA sugiere causas, humano entiende contexto |
|                                                                                                 | Revisión seguridad       | 10%  | 90%      | Humano auditor, IA checklist básico         |
|                                                                                                 | Optimización rendimiento | 50%  | 50%      | IA sugiere, humano mide y decide            |
|                                                                                                 |                          |      |          |                                             |
|  MANTENIMIENTO |                          |      |          |                                             |
|                                                                                                 | Refactorizar código      | 60%  | 40%      | IA propone, humano acepta/rechaza           |
|                                                                                                 | Actualizar dependencias  | 70%  | 30%      | IA identifica, humano prueba impacto        |
|                                                                                                 | Documentar funciones     | 80%  | 20%      | IA genera, humano revisa claridad           |
|                                                                                                 | Comentar código legacy   | 70%  | 30%      | IA explica, humano valida                   |

| Categoría     | Escenario                 | % IA | % Manual | Estrategia Recomendada           |
|---------------|---------------------------|------|----------|----------------------------------|
| 🔴 CRÍTICO     |                           |      |          |                                  |
|               | Autenticación             | 10%  | 90%      | Humano experto, IA solo consulta |
|               | Pagos / Transacciones     | 5%   | 95%      | Humano auditoría obligatoria     |
|               | Datos médicos             | 0%   | 100%     | NO USAR IA, regulación estricta  |
|               | Infraestructura cloud     | 30%  | 70%      | IA scripts, humano configuración |
|               | GDPR / Cumplimiento legal | 0%   | 100%     | Asesor legal humano              |
|               |                           |      |          |                                  |
| 🟢 APRENDIZAJE |                           |      |          |                                  |
|               | Aprender concepto nuevo   | 70%  | 30%      | IA explica, tú practicas         |
|               | Entender código ajeno     | 80%  | 20%      | IA documenta, tú aprendes        |
|               | Explorar tecnologías      | 60%  | 40%      | IA compara, tú pruebas           |

# IA vs Humano en el Desarrollo de Software





# EJEMPLOS PRÁCTICOS



## Ejemplo 1: Formulario de registro

### # ANÁLISIS:

- # - Datos personales: SÍ (email, nombre) → ⚠ Precaución
- # - Crítico: NO (no es pago)
- # - Repetitivo: SÍ (validaciones típicas)

### # DECISIÓN: ● IA + revisión

- # - IA: genera validaciones básicas
- # - TU: revisas, anonimizas datos en prompts
- # - TU: auditoría final

## Ejemplo 3: Script interno de migración

### # ANÁLISIS:

- # - Datos sensibles: NO (datos anonimizados)
- # - Crítico: MEDIO (una vez, puede fallar)
- # - Repetitivo: SÍ (transformaciones típicas)

### # DECISIÓN: ● 50% IA

- # - IA: genera script base
- # - TU: pruebas con copia de seguridad
- # - TU: supervisas ejecución

## Ejemplo 2: Pasarela de pago

### # ANÁLISIS:

- # - Datos sensibles: SÍ (tarjetas)
- # - Crítico: SÍ (dinero real)
- # - Repetitivo: NO (cada integración es única)

### # DECISIÓN: ● NO USAR IA para código final

- # - IA: solo para aprender sobre APIs de pago
- # - TU: implementas manualmente
- # - Auditor externo: revisa seguridad



## Uso de IA en el Desarrollo de Software



### USA IA CUANDO:

- Tareas repetitivas y conocidas
- Prototipos y exploración
- Generar estructura base
- Aprender conceptos nuevos
- Documentar código existente

### USA IA CON PRECAUCIÓN CUANDO:

- Lógica de negocio específica
- Código con datos personales (anonimizar)
- Optimizaciones (validar resultados)
- Refactorizaciones (revisar cambios)

### NO USES IA CUANDO:

- Datos extremadamente sensibles
- Sistemas críticos de seguridad
- Código con licencia restrictiva
- No entiendes lo que estás haciendo
- La responsabilidad legal es tuya y no puedes delegarla

### REGLA DE ORO:

- ☐ "IA acelera, pero TÚ respondes"
- ☐ "IA sugiere, pero TÚ decides"
- ☐ "IA escribe, pero TÚ entiendes"

### 4.5 Errores comunes en la colaboración humano-IA

- Depender demasiado de la IA
- Aceptar código sin entenderlo
- No revisar seguridad
- No validar rendimiento
- No documentar qué partes generó la IA
- No pedir explicaciones
- No dividir el trabajo en pasos

## 1. Depender demasiado de la IA

Error: dejar que la IA tome decisiones críticas sin supervisión.

Ejemplo: La IA genera una consulta SQL compleja para un informe financiero.



```
SELECT * FROM transactions WHERE date > '2024-01-01';
```

### Problema:

- No filtra por usuario
- No controla permisos
- No limita resultados
- Puede exponer datos sensibles

## 2. Aceptar código sin entenderlo

Error: copiar y pegar código que “parece correcto”.

Ejemplo: La IA genera una función de autenticación

```
if (user && user.password === hash(inputPassword)) {
 return true;
}
```

### Problema:

- No hay salting
- No hay rate limiting
- No hay logs
- No hay manejo de errores

### 3. No revisar seguridad

Error: asumir que la IA siempre genera código seguro.

Ejemplo: La IA genera un endpoint:



```
app.post("/upload", upload.any(), (req, res) => {
 saveFile(req.files[0]);
 res.send("OK");
});
```

#### Problema:

- No valida tipo de archivo
- No limita tamaño
- No evita path traversal
- No revisa permisos

### 4. No validar rendimiento

Error: aceptar soluciones que funcionan, pero son lentas o ineficientes.

Ejemplo: La IA propone:

```
const unique = array.filter((x, i) => array.indexOf(x) === i);
```

#### Problema:

- Complejidad  $O(n^2)$
- En listas grandes es un desastre

Alternativa eficiente:

```
const unique = [...new Set(array)];
```

## 5. No documentar qué partes generó la IA

**Error:** mezclar código humano e IA sin dejar rastro.

### **Ejemplo:**

Un módulo entero fue generado por IA, pero nadie lo sabe. Meses después, otro desarrollador intenta modificarlo y no entiende por qué está estructurado así.

**Lección:** Documentar evita confusión, facilita auditorías y mejora la trazabilidad.

## 6. No pedir explicaciones

**Error:** no aprovechar la IA como herramienta de aprendizaje.

### **Ejemplo:**

La IA genera un algoritmo de ordenación personalizado. El alumno lo acepta sin preguntar:

"Explícame este algoritmo paso a paso."

**Lección:** La IA no solo genera código: **puede enseñar**, pero solo si se lo pides.

## 7. No dividir el trabajo en pasos

**Error:** pedir todo en un único prompt gigante.

### **Ejemplo:**

"Hazme una API completa con autenticación, base de datos, tests y documentación."

La IA genera algo enorme, difícil de revisar y lleno de suposiciones incorrectas.

**Lección:** El trabajo profesional se hace **por iteraciones**, no por megasolicitudes.

## ■ 4.6 Buenas prácticas avanzadas

- Pedir a la IA que explique su propio código
- Pedir alternativas de implementación
- Pedir análisis de complejidad
- Pedir mejoras de seguridad
- Pedir refactorización incremental
- Pedir tests antes y después
- Usar la IA como "pair programmer"

# El modelo de colaboración efectiva

## Proceso de Mejora de Código con IA



## 1. PEDIR QUE LA IA EXPLIQUE SU PROPIO CÓDIGO

### Mal enfoque

```
prompt = "Hazme un QuickSort"
Usas el código sin entenderlo
```

### Buen enfoque

```
PASO 1: Generar código
prompt = "Implementa QuickSort en Python"

PASO 2: Pedir explicación
prompt = """
Ahora explícame este código línea por línea:
- ¿Qué hace cada parte?
- ¿Por qué elegiste esta implementación?
- ¿Dónde está la recursión?
- ¿Qué pasa en el caso base?
"""
```

# 1. PEDIR QUE LA IA EXPLIQUE SU PROPIO CÓDIGO

## Ejemplo real

```
Código generado por IA
def quicksort(arr):
 if len(arr) <= 1:
 return arr
 pivot = arr[len(arr) // 2]
 left = [x for x in arr if x < pivot]
 middle = [x for x in arr if x == pivot]
 right = [x for x in arr if x > pivot]
 return quicksort(left) + middle + quicksort(right)
```

# Pides explicación y IA responde:

"""

EXPLICACIÓN LÍNEA POR LÍNEA:

1. `def quicksort(arr):`  
Define la función que recibe una lista 'arr'
  2. `if len(arr) <= 1:`  
CASO BASE: Si la lista tiene 0 o 1 elemento, ya está ordenada  
Porque una lista de 1 elemento siempre está ordenada
  3. `pivot = arr[len(arr) // 2]`  
Elegimos el elemento del medio como pivote  
`len(arr)//2` es la posición central  
Esto ayuda a evitar el peor caso en listas ya ordenadas
  4. `left = [x for x in arr if x < pivot]`  
List comprehension: crea lista con elementos MENORES que pivote
  5. `middle = [x for x in arr if x == pivot]`  
Elementos IGUALES al pivote (maneja duplicados)
  6. `right = [x for x in arr if x > pivot]`  
Elementos MAYORES que pivote
  7. `return quicksort(left) + middle + quicksort(right)`  
RECUSIÓN: ordena left y right, y combina con middle  
¡Ojo! Esto crea nuevas listas en cada llamada (no in-place)
- """



## 2. PEDIR ALTERNATIVAS DE IMPLEMENTACIÓN



ALTERNATIVA 2: Con dict + lista doblemente enlazada manual

```
"""

class Node:
 def __init__(self, key, val):
 self.key = key
 self.val = val
 self.prev = None
 self.next = None

class LRUCache_Manual:
 def __init__(self, capacity):
 self.capacity = capacity
 self.cache = {}
 self.head = Node(0, 0)
 self.tail = Node(0, 0)
 self.head.next = self.tail
 self.tail.prev = self.head

... (implementación completa)

✅ Ventajas: Control total, más rápido en teoría
❌ Desventajas: Más código, propenso a errores
📊 Complejidad: O(1) real
```

ALTERNATIVA 3: Con biblioteca externa (cachetools)

```
"""

from cachetools import LRUCache

class LRUCache_Biblioteca:
 def __init__(self, capacity):
 self.cache = LRUCache(maxsize=capacity)

 def get(self, key):
 return self.cache.get(key, -1)

 def put(self, key, value):
 self.cache[key] = value

✅ Ventajas: Probado, feature-rich, mantenido
❌ Desventajas: Dependencia externa
📊 Complejidad: O(1) garantizado
"""
```

## 💡 2. PEDIR ALTERNATIVAS DE IMPLEMENTACIÓN

```
prompt = """
```

```
Tengo que implementar un caché LRU.
```

```
Dame 3 alternativas de implementación con:
```

- Ventajas de cada una
- Inconvenientes
- Cuándo usar cada una
- Complejidad Big O

```
"""
```

```
"""
ALTERNATIVA 1: Con OrderedDict (Python puro)
```

```
"""
```

```
from collections import OrderedDict
```

```
class LRUCache_OrderedDict:
```

```
 def __init__(self, capacity):
```

```
 self.cache = OrderedDict()
```

```
 self.capacity = capacity
```

```
 def get(self, key):
```

```
 if key not in self.cache:
```

```
 return -1
```

```
 self.cache.move_to_end(key)
```

```
 return self.cache[key]
```

```
 def put(self, key, value):
```

```
 if key in self.cache:
```

```
 self.cache.move_to_end(key)
```

```
 self.cache[key] = value
```

```
 if len(self.cache) > self.capacity:
```

```
 self.cache.popitem(last=False)
```

```
✅ Ventajas: Simple, legible, usa stdlib
```

```
❌ Desventajas: OrderedDict tiene overhead
```

```
📊 Complejidad: O(1) amortizado
```

```
"""
```

### ⚡ 3. PEDIR ANÁLISIS DE COMPLEJIDAD

## Respuesta IA

```
código = """
def buscar_duplicados(lista):
 for i in range(len(lista)):
 for j in range(i+1, len(lista)):
 if lista[i] == lista[j]:
 return True
 return False
"""

prompt = f"""
Analiza la complejidad temporal y espacial de este código:
{código}

Además:
- Identifica cuellos de botella
- Sugiere optimizaciones
- Compara con alternativas
"""
```

```
"""
```

### ANÁLISIS DE COMPLEJIDAD:



#### TEMPORAL:

- Peor caso:  $O(n^2)$  cuando no hay duplicados  
Dos bucles anidados:  $n * (n-1)/2$  operaciones
- Mejor caso:  $O(1)$  si el primer par ya es duplicado
- Caso promedio:  $O(n^2/2) \approx O(n^2)$



#### ESPACIAL:

- $O(1)$  - solo usa variables contadoras
- No crea estructuras adicionales



#### CUELLOS DE BOTELLA:

- El bucle anidado es el problema principal
- Ineficiente para listas grandes (>1000 elementos)

## ⚡ 3. PEDIR ANÁLISIS DE COMPLEJIDAD

### 🚀 OPTIMIZACIONES SUGERIDAS:

#### 1. Usar conjunto (HashSet) - $O(n)$ :

```
"""
def buscar_duplicados_optimo(lista):
 vistos = set()
 for elemento in lista:
 if elemento in vistos:
 return True
 vistos.add(elemento)
 return False
"""
```

✅ Ventaja:  $O(n)$  tiempo,  $O(n)$  espacio

❌ Desventaja: Usa más memoria

#### 2. Ordenar primero - $O(n \log n)$ :

```
"""
def buscar_duplicados_ordenando(lista):
 lista_ordenada = sorted(lista)
 for i in range(len(lista_ordenada)-1):
 if lista_ordenada[i] == lista_ordenada[i+1]:
 return True
 return False
"""
```

✅ Ventaja: Modifica la lista (o copia)

❌ Desventaja: Más lento que HashSet

### 📌 RECOMENDACIÓN:

Para la mayoría de casos, usar la versión con HashSet a menos que la memoria sea extremadamente limitada.

"""

## 4. PEDIR MEJORAS DE SEGURIDAD

```
codigo_inseguro = """
def login(username, password):
 query = f"SELECT * FROM users WHERE user='{username}' AND pass='{password}'"
 result = db.execute(query)
 return len(result) > 0
"""
```

```
prompt = f"""
Este código tiene problemas de seguridad.
{codigo_inseguro}
```

Identifica:

- Vulnerabilidades específicas
- Cómo explotarlas
- Cómo solucionarlas
- Mejores prácticas

## Respuesta IA

### VULNERABILIDADES IDENTIFICADAS:

#### 1. SQL INJECTION (CRÍTICA)

- El código concatena strings directamente
- Un atacante puede poner: ' OR '1'='1
- Resultado: SELECT \* FROM users WHERE user='' OR '1'='1' AND...
- ¡Acceso sin contraseña!

#### 2. CONTRASEÑAS EN TEXTO PLANO

- Se comparan directamente con la BD
- Deberían estar hasheadas

#### 3. FALTA DE VALIDACIÓN

- No se valida el input
- Acepta cualquier cosa

#### 4. EXPOSICIÓN DE ERRORES

- No hay manejo de excepciones
- Podría mostrar errores internos



## ● SOLUCIÓN COMPLETA:

"""

```
import hashlib
import secrets
from typing import Optional
```

```
class LoginSeguro:
```

```
 def __init__(self, db):
 self.db = db
```

```
 def _hash_password(self, password: str, salt: Optional[str] = None) -> tuple:
```

```
 """Genera hash seguro de contraseña"""
```

```
 if salt is None:
```

```
 salt = secrets.token_hex(16)
```

```
 # Usar PBKDF2 (resistente a fuerza bruta)
```

```
 key = hashlib.pbkdf2_hmac(
```

```
 'sha256',
```

```
 password.encode('utf-8'),
```

```
 salt.encode('utf-8'),
```

```
 100000 # Iteraciones
```

```
)
```

```
 return salt, key.hex()
```

```
 def login(self, username: str, password: str) -> bool:
```

```
 # 1. Validar entrada
```

```
 if not username or not password:
```

```
 return False
```

```
 # 2. Sanitizar (aunque usamos parámetros)
```

```
 username = username.strip()
```

```
 # 3. Consulta segura con parámetros
```

```
 query = "SELECT salt, password_hash FROM users WHERE username = %s"
```

```
 try:
```

```
 # Usar parámetros, NO concatenar
```

```
 result = self.db.execute(query, (username,))
```

```
 if not result:
```

```
 return False
```

```
 salt, stored_hash = result[0]
```

```
 # 4. Verificar contraseña
```

```
 _, computed_hash = self._hash_password(password, salt)
```

```
 # 5. Comparación segura (tiempo constante)
```

```
 return secrets.compare_digest(computed_hash, stored_hash)
```

```
 except Exception as e:
```

```
 # Log seguro (sin exponer detalles)
```

```
 self.logger.error(f"Error en login para usuario: {username}")
```

```
 return False
```

```
 """
```

### ✅ MEJORAS IMPLEMENTADAS:

- No SQL injection (parámetros)
- Passwords hasheadas con salt
- Validación de entrada
- Manejo de errores seguro
- Logging sin exponer datos
- Comparación en tiempo constante

```
 """
```

## 5. PEDIR REFACTORIZACIÓN INCREMENTAL

```
codigo_original = """
def process_data(d):
 r = []
 for x in d:
 if x > 10:
 r.append(x * 2)
 else:
 r.append(x * 3)
 return r
"""

prompt = f"""
Refactoriza este código en 3 pasos incrementales:
```

PASO 1: Mejorar nombres y legibilidad  
PASO 2: Añadir type hints y docstrings  
PASO 3: Optimizar rendimiento

Muéstrame cada paso por separado.

## Respuesta IA

```
"""
PASO 1: MEJORAR LEGIBILIDAD
"""

def process_data(data):
 results = []
 for value in data:
 if value > 10:
 results.append(value * 2)
 else:
 results.append(value * 3)
 return results

"""
```

```

"""
PASO 2: AÑADIR TIPADO Y DOCUMENTACIÓN
"""

from typing import List, Union

def process_data(data: List[Union[int, float]]) -> List[Union[int, float]]:
 """
 Procesa una lista de números aplicando un factor según su valor.

 Args:
 data: Lista de números a procesar

 Returns:
 Lista con valores transformados:
 - Si valor > 10: multiplicado por 2
 - Si valor <= 10: multiplicado por 3

 Example:
 >>> process_data([5, 15])
 [15, 30]
 """

 results: List[Union[int, float]] = []

 for value in data:
 factor = 2 if value > 10 else 3
 results.append(value * factor)

 return results

```

```

"""
PASO 3: OPTIMIZAR RENDIMIENTO
"""

from typing import List, Union
import numpy as np

def process_data(data: List[Union[int, float]]) -> List[Union[int, float]]:
 """
 Versión optimizada con list comprehension y numpy opcional

 # Opción A: List comprehension (rápida para listas pequeñas/medianas)
 return [x * (2 if x > 10 else 3) for x in data]

def process_data_numpy(data: Union[List, np.ndarray]) -> np.ndarray:
 """
 Versión super optimizada para grandes volúmenes con numpy

 data_array = np.array(data)
 return np.where(data_array > 10, data_array * 2, data_array * 3)

"""

```

📊 COMPARACIÓN RENDIMIENTO:

- Original: 100µs para 1000 elementos
- Paso 2: 95µs (similar, más legible)
- Paso 3: 30µs (list comp) / 5µs (numpy)





## 6. PEDIR TESTS ANTES Y DESPUÉS

100% 100% 100%

### Respuesta IA

```
prompt = """
```

Voy a refactorizar esta función:

```
def calculadora(a, b, op):
```

```
 if op == '+': return a + b
```

```
 if op == '-': return a - b
```

```
 if op == '*': return a * b
```

```
 if op == '/': return a / b
```

```
 return None
```

PRIMERO: Genera tests unitarios completos para la función actual

SEGUNDO: Dame una versión mejorada

TERCERO: Genera tests para la nueva versión

```
"""
```

```
"""
```

TESTS PARA VERSIÓN ORIGINAL

```
"""
```

```
import pytest
```

```
def test_calculadora_original():
```

```
 # Tests básicos
```

```
 assert calculadora(2, 3, '+') == 5
```

```
 assert calculadora(5, 2, '-') == 3
```

```
 assert calculadora(4, 3, '*') == 12
```

```
 assert calculadora(10, 2, '/') == 5
```

```
 # Tests casos edge
```

```
 assert calculadora(0, 5, '+') == 5
```

```
 assert calculadora(5, 0, '-') == 5
```

```
 assert calculadora(0, 5, '*') == 0
```

```
 assert calculadora(5, 0, '/') == float('inf') # División por cero
```

```
 # Tests operador inválido
```

```
 assert calculadora(2, 3, '%') is None
```

```
 # Tests tipos
```

```
 assert calculadora(2.5, 1.5, '+') == 4.0
```

```
 assert calculadora(-2, -3, '+') == -5
```

```
print("✅ Tests originales listos")
```

```
"""
```

## 📦 VERSIÓN MEJORADA

```
"""
```

```
from typing import Union, Optional
import logging
```

```
def calculadora_mejorada(
 a: Union[int, float],
 b: Union[int, float],
 operacion: str
) -> Optional[Union[int, float]]:
```

```
 """
```

Calculadora con manejo de errores y logging

Args:

a: Primer número  
b: Segundo número  
operacion: '+', '-', '\*', '/'

Returns:

Resultado de la operación o None si error

Raises:

ValueError: Si operación no válida

```
 """
```

# Validación de tipos

```
if not isinstance(a, (int, float)) or not isinstance(b, (int, float)):
 logging.error(f"Tipos inválidos: a={type(a)}, b={type(b)}")
 return None
```

```
operaciones = {
 '+': lambda: a + b,
 '-': lambda: a - b,
 '*': lambda: a * b,
 '/': lambda: a / b if b != 0 else float('inf')
}
```

```
if operacion not in operaciones:
 logging.warning(f"Operación no soportada: {operacion}")
 return None
```

```
try:
 resultado = operaciones[operacion]()
 logging.info(f"{a} {operacion} {b} = {resultado}")
 return resultado
except Exception as e:
 logging.error(f"Error en cálculo: {e}")
 return None
```

```
"""
```

## 🔧 TESTS PARA VERSIÓN MEJORADA

```
"""
```

```
def test_calculadora_mejorada():
 # Tests funcionalidad básica
 assert calculadora_mejorada(2, 3, '+') == 5
 assert calculadora_mejorada(5, 2, '-') == 3
 assert calculadora_mejorada(4, 3, '*') == 12
 assert calculadora_mejorada(10, 2, '/') == 5

 # Tests casos edge mejorados
 assert calculadora_mejorada(5, 0, '/') == float('inf')
 assert calculadora_mejorada(0, 5, '/') == 0

 # Tests validación
 assert calculadora_mejorada("2", 3, '+') is None # Tipo inválido
 assert calculadora_mejorada(2, 3, '%') is None # Operación inválida

 # Tests logging (verificar que no afecta resultado)
 import logging
 logging.basicConfig(level=logging.INFO)
 assert calculadora_mejorada(2, 3, '+') == 5 # Debe loggear pero funcionar

 print("📦 Tests mejorados listos")
```

```
"""
```

## 📊 COMPARACIÓN:

- Original: 8 tests, sin manejo errores
- Mejorada: 12 tests, logging, validación, typing
- Cobertura: 95% vs 70%

```
"""
```

## 7. USAR IA COMO "PAIR PROGRAMMER"

# 1. REVISIÓN DE CÓDIGO

```
prompt = """
```

```
Actúa como mi pair programmer senior.
```

```
Revisa este código como si fuéramos a hacer code review:
```

```
def procesar_pedidos(pedidos):
 total = 0
 for p in pedidos:
 if p['tipo'] == 'VIP':
 total += p['monto'] * 0.9
 else:
 total += p['monto']
 return total
```

```
Señala:
```

- Posibles bugs
- Mejoras de estilo
- Problemas de rendimiento
- Consideraciones de seguridad
- Cómo lo haría un experto

```
"""
```

# 2. BRAINSTORMING DE SOLUCIONES

```
prompt = """
```

```
Tenemos que implementar un sistema de notificaciones.
```

Opciones que consideramos:

- Email (SMTP)
- WebSockets
- Cola de mensajes (RabbitMQ)
- Webhooks

Como mi compañero de equipo:

- ¿Qué opinas de cada opción?
- ¿Qué no estamos considerando?
- ¿Cómo decidirías?
- ¿Qué patrones usarías?

```
"""
```



MINISTERIO  
DE EDUCACIÓN, FORMACIÓN PROFESIONAL  
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN  
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

## 7. USAR IA COMO "PAIR PROGRAMMER"

# 3. DEBUGGING COLABORATIVO

prompt = """

Este código falla aleatoriamente:

```
def procesar_archivos():
 for f in os.listdir('/tmp'):
 with open(f) as archivo:
 datos = archivo.read()
 # procesar datos...
```

Error: FileNotFoundError

Como mi pair programmer:

- ¿Por qué falla?
- ¿Cómo reproducirlo?
- ¿Cómo arreglarlo?
- ¿Cómo evitar estos errores?

"""

# 4. TOMA DE DECISIONES ARQUITECTÓNICAS

prompt = """

Estamos diseñando una API REST.

Contexto:

- 1000 peticiones/segundo
- Datos en tiempo real
- Equipo pequeño

¿Qué stack recomiendas?

- Framework: FastAPI vs Django vs Flask
- Base datos: PostgreSQL vs MongoDB
- Cache: Redis vs Memcached

Como mi compañero senior:

- Hazme preguntas para entender mejor
- Da pros/cons de cada opción
- Recomendación final con justificación

"""



MINISTERIO  
DE EDUCACIÓN, FORMACIÓN PROFESIONAL  
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN  
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

## 7. USAR IA COMO "PAIR PROGRAMMER"

🟡 🟠 ⬜

```
TÚ (preguntas como a un compañero)
prompt = """
Estoy implementando una función para buscar en un árbol binario.
¿Qué te parece este enfoque?

class Nodo:
 def __init__(self, valor):
 self.valor = valor
 self.izq = None
 self.der = None

def buscar(nodo, objetivo):
 if not nodo:
 return False
 if nodo.valor == objetivo:
 return True
 return buscar(nodo.izq, objetivo) or buscar(nodo.der, objetivo)
"""
```

```
IA (responde como compañero)
```

```
"""
```

|Buen enfoque básico! Como tu pair programmer, te sugiero:

👍 LO QUE FUNCIONA BIEN:

- Estructura simple y clara
- Caso base correcto (nodo vacío)
- Recursión apropiada

🔧 MEJORAS SUGERIDAS:

1. Si es un BST, podemos optimizar:

```
"""
```

```
def buscar_bst(nodo, objetivo):
 if not nodo:
 return False
 if nodo.valor == objetivo:
 return True
 elif objetivo < nodo.valor:
 return buscar_bst(nodo.izq, objetivo)
 else:
 return buscar_bst(nodo.der, objetivo)
"""
```

|Esto reduce la complejidad de  $O(n)$  a  $O(\log n)$ !

2. Considera iterativo para evitar recursión profunda:

```
"""
```

```
def buscar_iterativo(raiz, objetivo):
 nodo = raiz
 while nodo:
 if nodo.valor == objetivo:
 return True
 elif objetivo < nodo.valor:
 nodo = nodo.izq
 else:
 nodo = nodo.der
 return False
```

## 7. USAR IA COMO "PAIR PROGRAMMER"

3. Añade type hints y documentación:

"""

from typing import Optional

def buscar(nodo: Optional[Nodo], objetivo: int) -> bool:

"""

Busca un valor en el árbol binario.

💡 ALTERNATIVAS

Args:

nodo: Raíz del árbol (puede ser None)

objetivo: Valor a buscar

Returns:

True si existe, False en caso contrario

"""

# ... 🔧 REFACTORIZACIÓN

"""

📌 TESTS

📌 PREGUNTAS PARA REFLEXIONAR:

- ¿Qué tipo de árbol es? (BST, binario normal, etc.)

- ¿Hay valores repetidos?

- ¿Cuál es la profundidad máxima esperada?

¿Quieres que profundicemos en alguna de estas opciones?

"""

### Prompts

- "Explica este código línea por línea"• "¿Por qué tomaste esa decisión?"• "¿Qué pasa si cambia X?"
- "Dame 3 formas diferentes de hacer lo mismo"• "Compara ventajas y desventajas"• "¿Cuál usarías en producción?"
- "Analiza Big O temporal y espacial"• "Identifica cuellos de botella"• "Sugiere optimizaciones"
- "Busca vulnerabilidades"• "¿Cómo explotarías esto?"• "Mejora la seguridad"
- "Mejora este código paso a paso"• "Versión 1: legibilidad"• "Versión 2: rendimiento"• "Versión 3: mantenibilidad"
- "Genera tests primero"• "Cobertura para casos edge"• "Tests después de refactorizar"
- "¿Qué opinas de esta implementación?"• "Ayúdame a debuggear"• "¿Cómo abordarías este problema?"



| Categoría                                                                                            | Prompts                                                                                                                                                                                        |
|------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  EXPLICACIÓN        | <ul style="list-style-type: none"><li>• "Explica este código línea por línea"</li><li>• "¿Por qué tomaste esa decisión?"</li><li>• "¿Qué pasa si cambia X?"</li></ul>                          |
|  ALTERNATIVAS       | <ul style="list-style-type: none"><li>• "Dame 3 formas diferentes de hacer lo mismo"</li><li>• "Compara ventajas y desventajas"</li><li>• "¿Cuál usarías en producción?"</li></ul>             |
|  COMPLEJIDAD        | <ul style="list-style-type: none"><li>• "Analiza Big O temporal y espacial"</li><li>• "Identifica cuellos de botella"</li><li>• "Sugiere optimizaciones"</li></ul>                             |
|  SEGURIDAD          | <ul style="list-style-type: none"><li>• "Busca vulnerabilidades"</li><li>• "¿Cómo explotarías esto?"</li><li>• "Mejora la seguridad"</li></ul>                                                 |
|  REFACTORIZACIÓN    | <ul style="list-style-type: none"><li>• "Mejora este código paso a paso"</li><li>• "Versión 1: legibilidad"</li><li>• "Versión 2: rendimiento"</li><li>• "Versión 3: mantenibilidad"</li></ul> |
|  TESTS            | <ul style="list-style-type: none"><li>• "Genera tests primero"</li><li>• "Cobertura para casos edge"</li><li>• "Tests después de refactorizar"</li></ul>                                       |
|  PAIR PROGRAMMING | <ul style="list-style-type: none"><li>• "¿Qué opinas de esta implementación?"</li><li>• "Ayúdame a debuggear"</li><li>• "¿Cómo abordarías este problema?"</li></ul>                            |



Gracias.



MINISTERIO  
DE EDUCACIÓN, FORMACIÓN PROFESIONAL  
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN  
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN



